



VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
INSTITUTE OF COMPUTER SCIENCE

Software Architecture Project Report

Workout Progress Tracker

Done by:

Karolis Zabulis

signature

Supervisor:

Asst., Dr. Andrius Vytautas Misiukas
Misiūnas

Vilnius
2026

Contents

1	Project Description	3
2	Technologies Used	4
2.1	React Native, Expo, Nativewind	4
2.2	Laravel	4
2.3	PostgreSQL	4
3	Functionalities	4
4	Non-functional Requirements	5
5	UML Diagrams	6
5.1	Use Case Diagram	6
5.2	Activity Diagram	7
5.3	ER Model Diagram	8
5.4	Database Schema Diagram	9
6	Results	10
6.1	Changes During Development	10
6.1.1	Architectural Changes	10
6.1.2	Technological Changes	10
6.2	Deployment Guide	10
6.2.1	Backend Deployment	10
6.2.2	Mobile Application Deployment	11
6.3	Project Structure	11
6.3.1	Backend Structure	11
6.3.2	Mobile Application Structure	11
6.3.3	Web Application Structure	12
	Declaration of Artificial Intelligence	13
	References	14

1 Project Description

People that workout encounter a problem when you need to document your personal records. Information, such as lifted weight, repetition or set count, helps to track your individual progress. This mobile application creates an easy-to-use UI that allows you to document such information quickly and without much hassle. Additionally, the application helps you track your progress over a period of time.

2 Technologies Used

2.1 React Native, Expo, Nativewind

- React Native - technology that allows you to create mobile applications for iOS and android without the need to change the codebase.
- Expo - framework that makes mobile development faster and more convenient.
- Nativewind - framework that adds default responsiveness to your application.

2.2 Laravel

- Laravel - already familiar technology therefore, the development of the backend should be faster. It also gives you the ability to add a web application without much additional work to the backend.

2.3 PostgreSQL

- PostgreSQL - also a familiar technology and integrates well with Laravel.

3 Functionalities

- First iteration
 - User registration.
 - User session.
- Second iteration
 - Exercises CRUD.
 - Workout CRUD.
- Third iteration
 - Workout session CRUD.
 - Workout session and related data tracking.
 - Admin GUI (Web based interface, allows administration to insert public exercises and workouts).
- Fourth iteration
 - Local data caching without internet connection and data synchronization with connection.
 - Progress analysis charts based on criteria (based on lifted weight, based on repetitions done).

4 Non-functional Requirements

- Security - the application must be secure as all the data will be passed through the network.
- Usability - the application must be easy and fast to use as the main use case is to track your data during a workout.
- Scalability – program should be easy to scale up, this is one of the reasons why Laravel was used.

5 UML Diagrams

5.1 Use Case Diagram

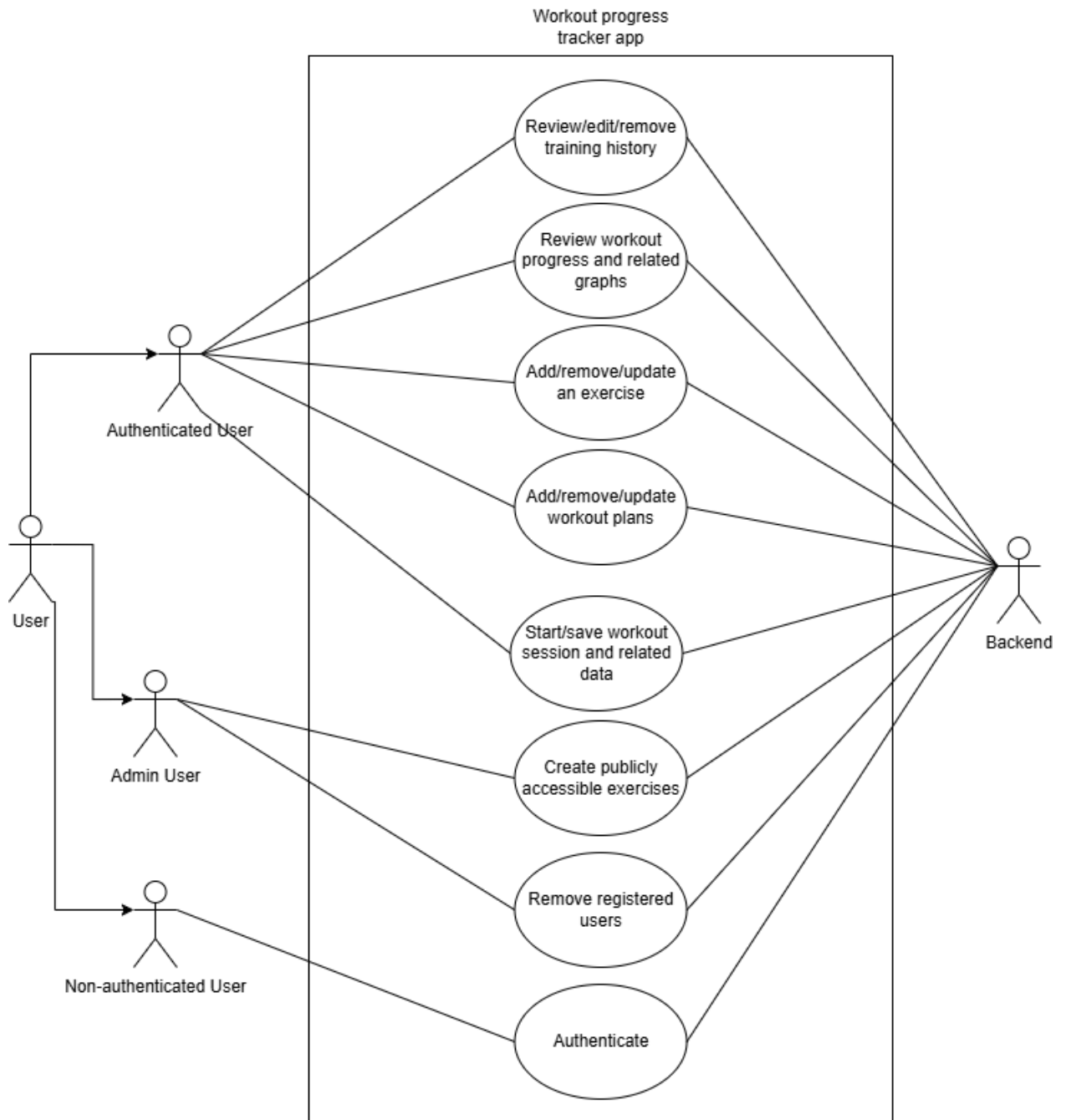


Figure 1. Use Case Diagram

The diagram displays functionalities of three user types, authenticated user, non-authenticated user and admin. Authenticated users have full capabilities of the application, while non-authenticated users have capabilities that only involve local data. Admins have special possibilities such as creating public workouts or removing user accounts.

5.2 Activity Diagram

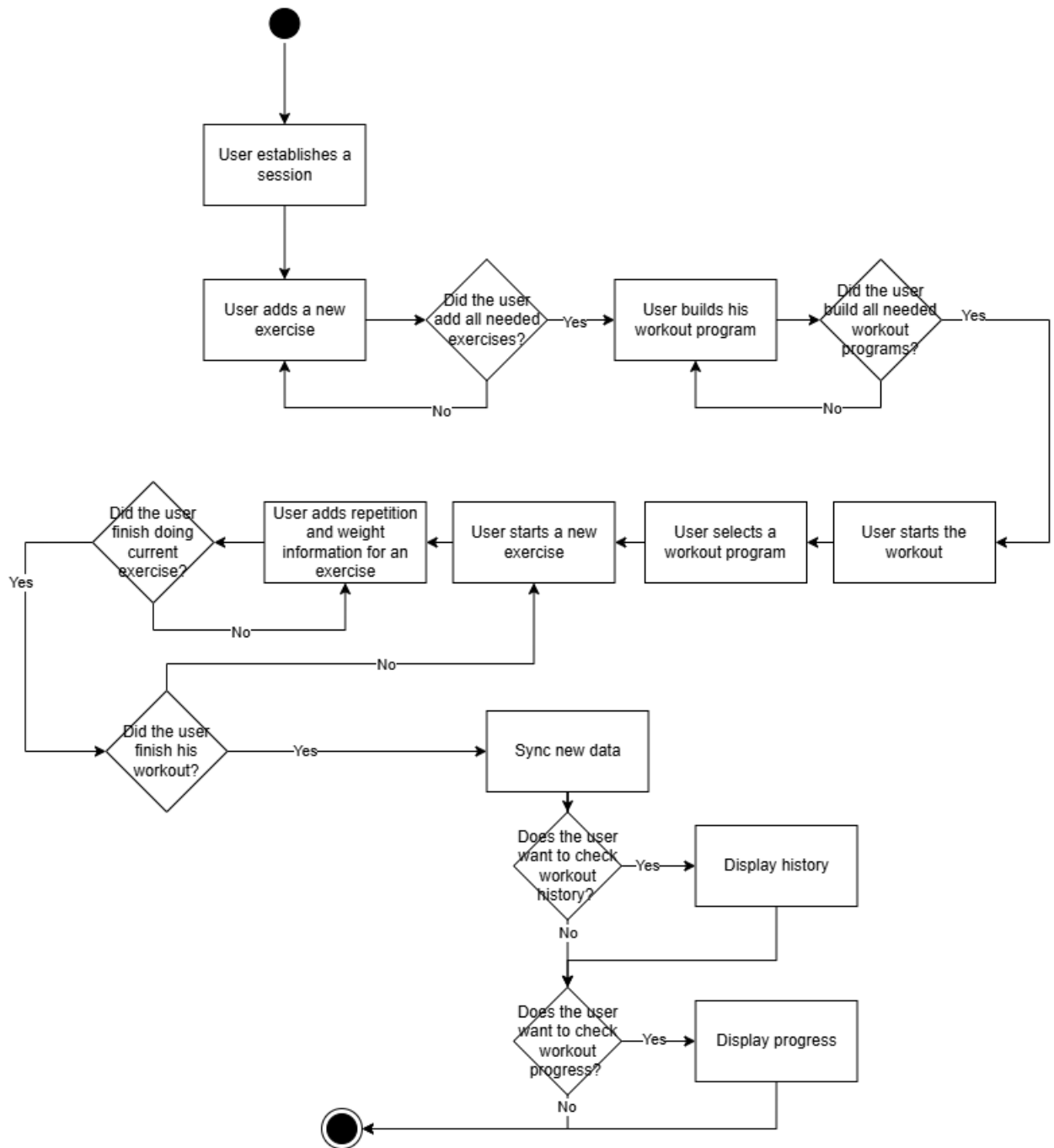


Figure 2. Activity Diagram

The activity diagram is displayed from a first-time authenticated user's perspective. Users have the ability to create their own exercises as well as workouts. The reasoning behind this decision and not using templates is that all workouts can be very different and include many different exercises. Therefore, it is nearly impossible to suit every user. Once the user builds his workout routines, he can start workout tracking, select a workout from his presets, and all that is left to do is to document the data. After a user finishes his workout, he can then preview his progress statistics or see the history of his workouts.

5.3 ER Model Diagram

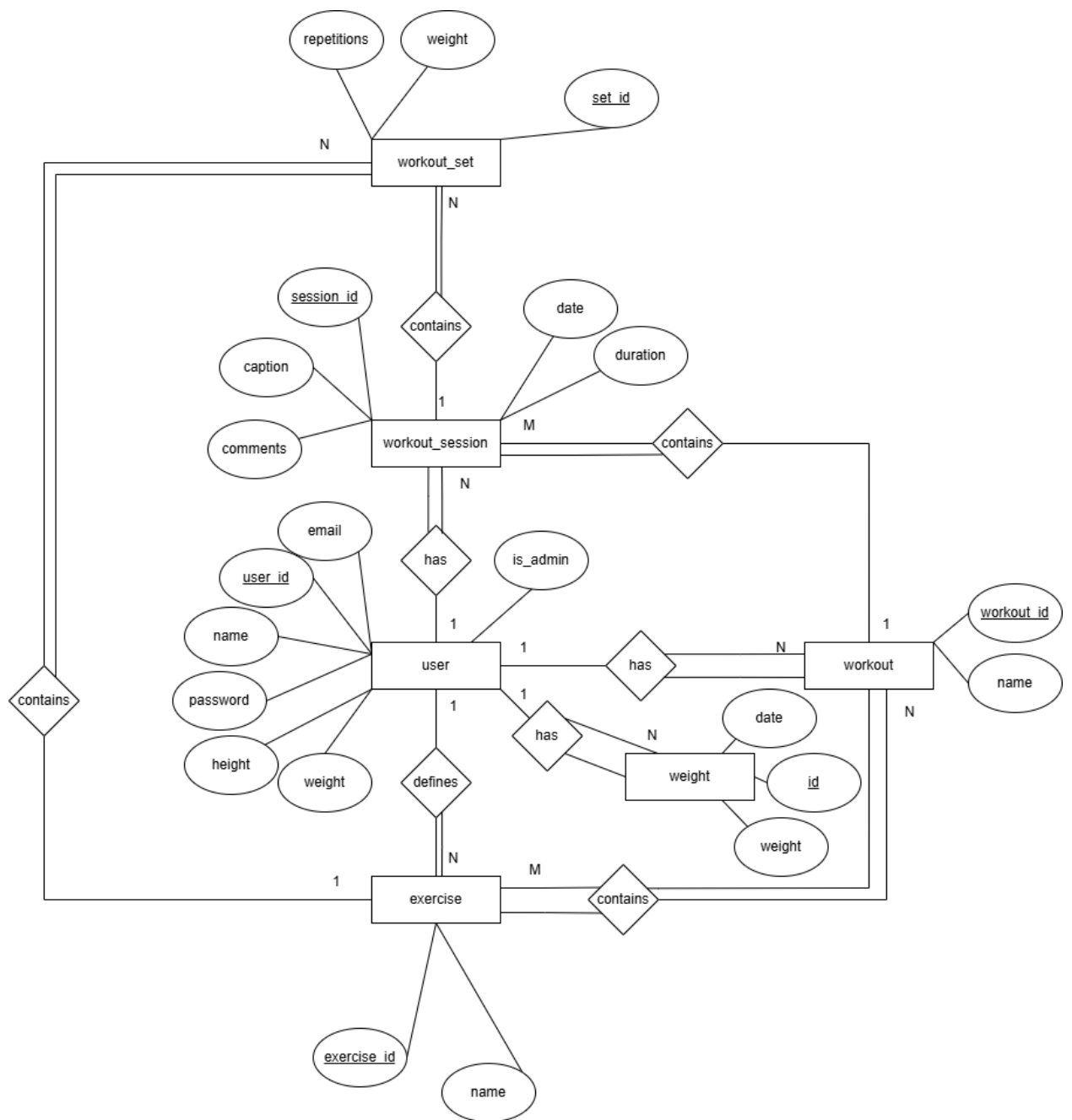


Figure 3. ER Model Diagram

The diagram sketches the example of the database tables. The user can create as many as he wants exercises and workouts, he can have many workout sessions. Exercise can repeat in different workouts as well as in workout sets. Workout contains many different exercises. Workout sessions can be with the same workout many different times, also it usually contains more than one set per exercise. Table weight is used for user's weight tracking and serves a history like purpose.

5.4 Database Schema Diagram

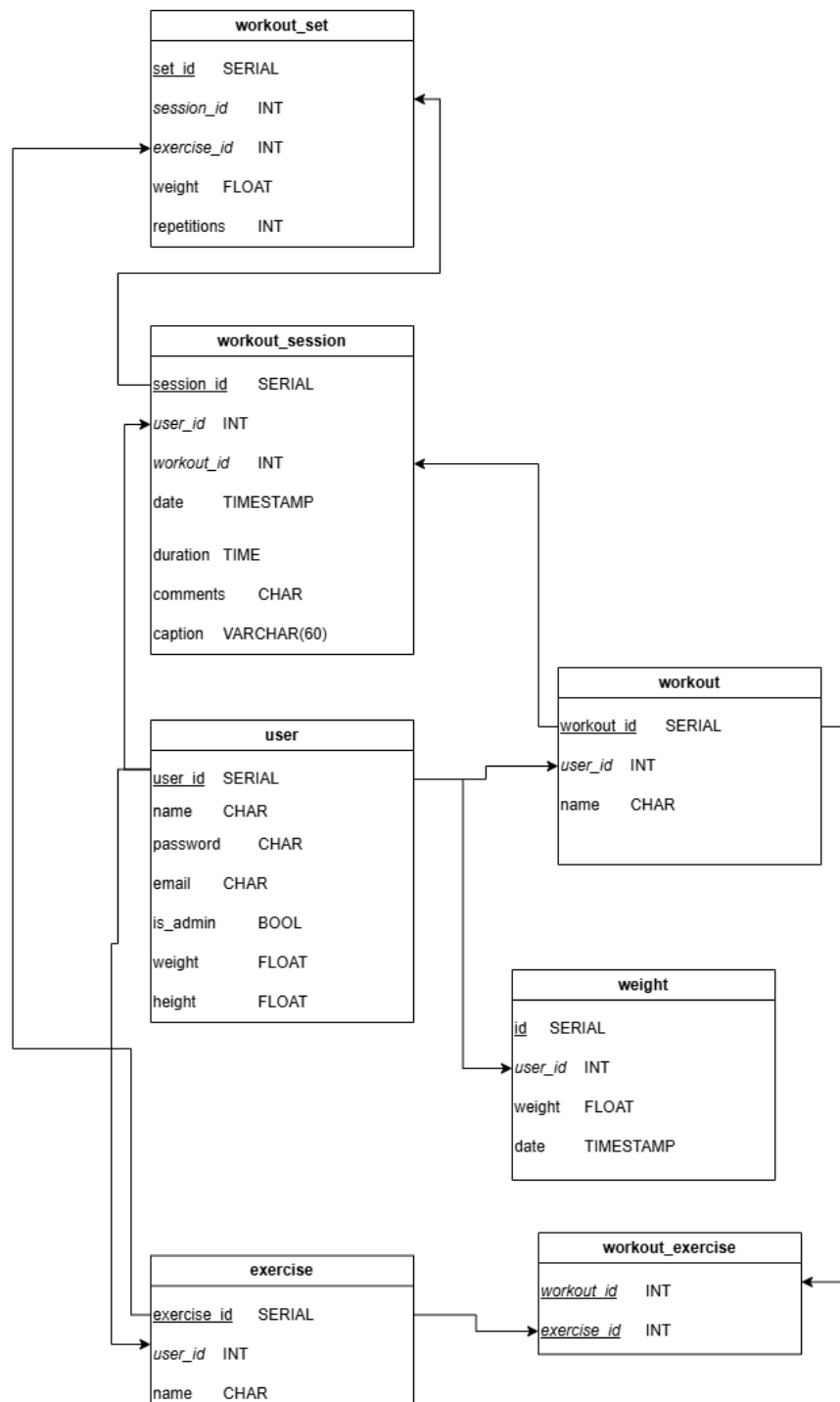


Figure 4. Database Schema Diagram

Summarizes the previous diagram by placing FKs in the right tables, also shows the variable types in each table.

6 Results

6.1 Changes During Development

Due to unforeseen limitations during the project planning period, the system required both architectural and technological changes.

6.1.1 Architectural Changes

- To create the possibility of displaying the user's weight change, a new database table was created called weight. It is used for storing the user's weight change.
- The table called user had its attributes expanded to house user specific information such as height and weight.
- Due to the creation of the admin role, the table called user, now stores a boolean attribute called is_admin.
- Due to the lack of time 4th iteration's 1st part, local data caching without internet connection and data synchronization with connection, was chosen not to implement.

6.1.2 Technological Changes

- During development it was noticed that React-Native does not have native chart support, thus additional JavaScript libraries, such as react-native-gifted-charts and react-native-chart-kit, were introduced.
- For the purpose of administration page development, Laravel Blade templates were introduced.

6.2 Deployment Guide

The system is made up of two main parts, one is the back-end and another is the mobile application. Both parts require different deployment procedures to work correctly. For the developer's convenience, procedures for both parts were semi-automated. This section discusses the steps required to successfully deploy the system. It is important to keep in mind that all of the automated scripts were written for Ubuntu-26-04 LTS operating system.

6.2.1 Backend Deployment

- When deploying the web application on a fresh machine, the user should execute a shell script, called setup_backend.sh, located in the scripts directory. This script installs required dependencies, creates 3 docker containers that are used by the system. After images are created, the script runs migrations, dependency installations and other required commands. Finally, the containers are restarted, so the previous updates can take action.
- For the ease of control, containers can be quickly shut down or started using a make file, which is located in the src/backend directory. The developer can simply write make up or make down and shut down or startup the containers respectively.

6.2.2 Mobile Application Deployment

- For mobile development framework called Expo was used which allows to compile and preview the application directly on the mobile device.
- When deploying the application on a fresh machine, the user should execute a script called `setup_frontend.sh` located in the scripts directory. This script installs required dependencies such as NodeJS, NPM and different JavaScript libraries utilized by the application. Once the script passes, the user should restart his shell environment for the installations to take action.
- To avoid the need for writing manual commands, the user can once again take advantage of the make file, which is located in the `src/frontend` directory. Executing `make up` will launch expo's development server. Once the server is launched a QR code as well as an IP address will be provided which then can be accessed using the expo go mobile application or with an android emulator, for more information refer expo's official documentation [1].

6.3 Project Structure

Project's structure can be divided into two different categories. First one is the backend which is responsible for request handling. The frontend covers both; a mobile application which is responsible for communicating with the backend and displaying the received information, and the administration's web application.

6.3.1 Backend Structure

- **Controllers:** Controllers ensure that incoming requests are valid, and the fields required are actually present. Other additional logic; such as calculations or data operations are also handled by the controller components.
- **Models:** Models work as an abstraction layer between a controller and the database, allowing them to easily interact with the database without leaving the controller.
- **Routes:** Routes work as endpoints allowing the user to send or receive data from the backend. Two different types of routes are used. The first type is API routes which are dedicated for the mobile application and return responses in JSON format. The other type is web routes; web routes are used for the administration page. They return rendered views.
- **Middleware:** Middleware is explicitly used to ensure that the user has required permissions to access a specific route. Specific use cases are as follows; is user an admin, is user authenticated, is user unauthenticated.
- **Migrations:** Migrations automate the creation of the database tables. They also take part in seeding the database user table with fake users.

6.3.2 Mobile Application Structure

- **Pages:** Mobile application pages are very similar to web application pages. They are dedicated to showing some specific display of components for the user. Usually, they involve some logic that is used for input validation or other computations as well as logic required to communicate with the application layer.

- **Components:** Several components make up a page. In those cases where the same component requires to have a specific styling and is used throughout several pages or several times in the same page, it is convenient to define it once and reuse it everywhere.
- **Contexts:** Contexts define interactions with the backend API. The main idea is to write specific interaction logic in one location and be able to reuse them. For example, Workout-Context is used for workout and exercise CRUDs, which are all located in different pages. By defining this logic in a context there is no need to rewrite the same logic in different pages and the created objects can be accessed from any page by importing the context.
- **Helpers:** These are the components that help to eliminate repetitive tasks but do not fit into any other categories. For example, in constants directory a file called colors exists, and it strictly defines the colors used throughout the project. One more example could be in lib directory axios is responsible for adding authentication tokens, if they exist, before sending requests.

6.3.3 Web Application Structure

- **Views:** Views are simple web pages that are developed using Laravel's Blade templates. They're used for the administration's web application.

Declaration of Artificial Intelligence

Artificial intelligence was used for coding assistance using Claude's Sonnet 4.6.

References

- [1] Expo's development environment setup.
<https://docs.expo.dev/get-started/set-up-your-environment/>.